
Natural Language Processing

Lecture-2 : Representing text with vector

❖ SUMMARY

What is a Word?

- Many definitions depending on **typography, syntax, semantics**, etc.
- **Semantic definition:**
 - Words are the **smallest linguistic expressions** with non-compositional meaning that can be used in isolation to convey semantic content (Stanford Encyclopedia of Philosophy).

Representing Text with Vectors

Objective:

- Given vocabulary $w_1, \dots, w_V$ and corpus D , associate **each word with a representation** that:
 - Identifies the word (**bijection**)
 - Captures **similarities** (morphology, syntax, semantics)
 - Helps solve **downstream NLP tasks**
- Vector-based representations = **embeddings**

1-Hot Encoding

- Traditional way: assign **unique integer** or **1-hot vector** to each word
 - Example:
 - movie = [1, 0, 0, 0, 0]
 - hotel = [0, 1, 0, 0, 0]
- Assumes word vectors are **orthonormal** (orthogonal + normalized)
- Problems:
 - Not informative: treats all words as **equidistant** (e.g., house – home = house – car)
 - Polysemy: same vector for multiple meanings (e.g., mouse computer vs animal)

Hand-Crafted Feature Representations

Potential features:

- Morphology: prefix, suffix, stem
 - Grammar: part of speech, gender, number
 - Shape: capitalization, digits, hyphens
 - Semantic relations to other words:
 - Synonyms, hypernyms, antonyms
 - Example: **WordNet**
-

WordNet

- **Word sense:** discrete representation of one aspect of a word's meaning
- **Lexical database** of word senses (English and other languages)
- Words grouped into **synsets** (cognitive synonym sets)
 - Example: S09293800 = {Earth, earth, world, globe}
- Polysemous words → multiple synsets
 - Example: S14867162 = {earth, ground}
- Synsets include **glosses** and are organized hierarchically (IS-A relation)
 - Example: apple IS-A fruit

WordNet Relations

- **Hyponym:** X is an instance of Y (cat → hyponym of animal)
- **Hypernym:** Y is an instance of X (animal → hypernym of cat)
- **Co-hyponyms:** share the same hypernym (cat and dog)
- **Meronym:** X is part of Y (wheel → meronym of car)
- **Holonym:** Y is part of X (car → holonym of wheel)

Similarity in WordNet

- **Synset similarity:** shorter hypernym/hyponym path → higher similarity
- **Word similarity example:** $\text{sim}(\text{dog}, \text{cat})$ depends on path in hierarchy

Term	Definition	Example
Word Sense	A specific meaning of a word in a particular context.	<i>Bank</i> → financial institution / river edge
Gloss	A short definition or explanation of a word sense.	<i>Bank (financial)</i> → “a financial institution that accepts deposits and channels the money into lending activities”
WordNet	A structured lexical database of word senses, their synonyms (synsets), and semantic relationships.	Synset for <i>car</i> : {car, auto, automobile, machine, motorcar} with gloss “a motor vehicle with four wheels”

Word and Synset Similarity

Similarity between two synsets S_1 and S_2 :

$$\text{sim}(S_1, S_2) = \frac{1}{\text{length}(\text{path}(S_1, S_2))}$$

Similarity between two words w_1 and w_2 :

$$\text{sim}(w_1, w_2) = \max_{\substack{S_1, S_2 \\ w_1 \in S_1 \\ w_2 \in S_2}} \text{sim}(S_1, S_2)$$

Idea: Uses **WordNet hierarchy** (hypernym/hyponym paths) to compute similarity.

Subjectivity vs Objectivity

- **Subjective statements:** express **personal opinions, feelings, beliefs**
 - **Objective statements:** describe **verifiable facts**
-

Distributional Hypothesis (Firth, 1957)

- Quote: “You shall know a word by the company it keeps.”
- Idea: **Words appearing in similar contexts have similar meanings**
- Enables building **vector representations** of words based on **their surrounding words**.

Steps for Distributional Word Representation:

1. Define the **context** of a word
 2. Count occurrences of the target word in that context
 3. Build vectors from (a function of) these context occurrence counts
- Example: Meaning of “**bardiwac**” inferred from context sentences:
 - Context shows **bardiwac = heavy red alcoholic beverage made from grapes**

Contextual Word Embeddings

- Formula:
$$x_w = f(w, \text{Context}(w))$$
 - Components:
 - x_w = **contextual embedding** of word w
 - w = target word
 - $\text{Context}(w)$ = surrounding words
 - f = mapping function (e.g., Transformer, Bi-LSTM)
 - **Importance:**
 - Captures **polysemy** (multiple meanings)
 - Example:
 - “I sat on the **bank** of the river.” → embedding reflects river bank
 - “I deposited money at the **bank**.” → embedding reflects financial bank
-

Defining Context

- Can be:
 - Surrounding words (left & right)
 - All words in sentence or paragraph
 - Words after preprocessing/filtering

Modeling Context

- **Approach 1: Count-Based**
 - Measure **frequency** of words in context for each vocabulary word
 - Build vector representations from these counts
- **Approach 2: Prediction-Based**
 - Use models to **predict target words from context** (not detailed in this slide yet)

Counting Word Occurrences in Context

- Build a **co-occurrence matrix**: counts of how often each word appears in the **context of another word** (e.g., “dog”).
- **Naïve approach**: take the row of the co-occurrence matrix as the word vector.

Limits of Raw Co-Occurrence

- Representations depend heavily on **corpus size**
 - Frequent words dominate representations
 - Very sensitive to changes in **rare words**
-

Pointwise Mutual Information (PMI)

- Idea: use **relative probabilities** of co-occurrence instead of absolute counts
- Intuition:
 - Words that are **dependent** (e.g., “dog” and “cat”) $\rightarrow P(dog, cat) \approx P(dog)P(cat) \rightarrow$ smaller PMI
- Word vectors = **row of PMI matrix**
 - Use **Positive PMI** (negative PMI set to 0)
 - Apply **Laplace smoothing** for unseen pairs
 - PMI **normalized**, less sensitive to frequent words

Limits of PMI

- Matrix size is **$O(V^2)$** $\rightarrow$ very large for big vocabularies
 - Large vectors are hard to use in practice (e.g., 1M words)
 - Cannot compare word vectors from different corpora unless **vocabulary is identical**
-

Dimensionality Reduction with SVD

- **Goal**: reduce large PMI-based vectors to smaller, manageable embeddings
- Steps:
 - Build **symmetric definite matrix** from PMI
 - Decompose with **Singular Value Decomposition (SVD)**
 - $U \in \mathbb{R}^{V \times d}$ gives **word embeddings in latent space**
- Properties:
 - U is **orthonormal**
 - Captures **highest variance** of original embeddings

$$P = U_d \Sigma_d V_d^T$$

Limits of SVD

- Still need to store **$O(V^2)$ matrix**
 - Computation: **$O(V * d^2)$** → inefficient for very large vocabularies
 - Build very large matrix for reducing
-

Solution: Prediction-Based Word Embeddings

- Idea: **learn embeddings directly** from predicting words from context, avoiding large matrices
- Combines vector learning and corpus statistics **simultaneously**

Core Philosophy of Prediction-Based Models

- Shift from **counting occurrences** (co-occurrence matrices) to **predicting context words**.
- Learn **dense, low-dimensional word vectors directly** instead of building large sparse matrices + dimensionality reduction.

1. Learning Dense Word Vectors

- Older methods: sparse vectors (length = vocabulary size, mostly zeros).
- Prediction-based: start with **small, random dense vectors** (e.g., 300 dimensions).
- Vectors are updated iteratively to accurately capture **word meaning**.

2. Distributional Hypothesis

- Built on Firth's principle: "You shall know a word by the company it keeps."
- Words with **similar contexts** (e.g., "coffee" & "tea") have **similar vectors**.

3. Parameterizing Words as Dense Vectors

- Every word = **learnable parameters (weights)** in high-dimensional space.
- Each dimension captures **implicit features**: gender, tense, formality, etc.
- No manual feature engineering required.

4. Learning to Predict Context

- The "engine" of the model: predict surrounding words to improve vectors.

Two main approaches:

- **Skip-Gram**: target word → predict surrounding context words
 - **CBOW (Continuous Bag of Words)**: context words → predict missing target word
-

Word2Vec Skip-Gram Model

- Representations:
 - **$W \in \mathbb{R}^{V \times d}$** (focus word embeddings)
 - **$C \in \mathbb{R}^{V \times d}$** (context word embeddings)
- For sequence $(w_1, \dots, w_T)$:
 - Pick a **focus word** $w \rightarrow$ vector **w**
 - Pick a **context word** $c \rightarrow$ vector **c**

- Maximize **log-likelihood**: $\max_{w,c} \log p(c | w)$
- Intuition: **classification problem** over a huge label space ($V \sim 100K$)

Softmax Probability

$$p(c | w) = \frac{e^{w \cdot c}}{\sum_v e^{w \cdot v}}$$

$$\log p(c | w) = w \cdot c - \log \sum_v e^{w \cdot v}$$

Softmax is computationally expensive → use **Negative Sampling**

Negative Sampling

- Instead of computing probability over entire vocabulary:
 - Sample $K \ll V$ **negative words** not in context
 - New objective: only update vectors for target + K negatives

$$\sigma(x, y) = \frac{1}{1 + e^{-x \cdot y}}$$

$$\log \sigma(w, c) + \frac{1}{K} \sum_{v \in N_K} \log \sigma(-w, v)$$

Complexity: $O(K)$ independent of V

- Allows **scaling to billion-token datasets**
-

Skip-Gram Training Algorithm

Given: corpus C , vocab V , negative samples K , window size l , vector dimension d , learning rate α_t

1. Initialize $\mathbf{W}$, $\mathbf{C}$ randomly
2. Loop over dataset for E epochs:
 - Sample sentence $s = (w_1, \dots, w_n)$
 - Sample focus-context pair (w_i, w_j) with $|i - j| \leq l$
 - Sample negative words $N_K \subset V$
 - Compute loss:

$$l(\mathbf{W}, \mathbf{C}) = -\sigma(\mathbf{w}, \mathbf{c}) - \frac{1}{K} \sum_{v \in N_K} \log \sigma(-\mathbf{w}, \mathbf{v})$$

- Update parameters with **SGD**:

$$\begin{aligned} \mathbf{W}_t &= \mathbf{W}_{t-1} - \alpha_t \nabla l(\mathbf{W}_{t-1}, \mathbf{C}_{t-1}) \\ \mathbf{C}_t &= \mathbf{C}_{t-1} - \alpha_t \nabla l(\mathbf{W}_{t-1}, \mathbf{C}_{t-1}) \end{aligned}$$

Key advantages:

- No memory bottleneck
 - Scales to **very large corpora**
 - Efficient thanks to negative sampling + SGD
-

SGNS and PMI Connection

- Embedding matrices estimate **Shifted PMI**:

$$M_{ij}^{SGNS} = W_i \cdot C_j = \text{PMI}(w_i, c_j) - \log k$$

- Explains why **SGNS works similarly to PMI-based embeddings** in practice.

Practical Notes on Word2vec

- Works well with **Deep Learning architectures** (e.g., LSTMs for NER)
 - Variants: CBOW, GloVe, fastText (multilingual, subword)
 - **Limitations:**
 - Fixed vocabulary → cannot handle new words easily
 - Polysemy → one vector per word, cannot capture multiple meanings
 - No morphology modeling
-

Evaluation of Word Embeddings

1. Types of Evaluation

- **Extrinsic Evaluation:**
 - Use embeddings in a **task-specific model** (e.g., NER, sentiment analysis)
 - Measure performance on the task → shows practical usefulness
- **Intrinsic Evaluation:**
 - Check if embeddings **reflect linguistic similarity** independent of a task

2. Defining “Similarity”

Embeddings should place **similar words close together**. Types of similarity:

- **Morphological:** e.g., computer ~ computers
 - **Syntactic:** e.g., determiners (the, a)
 - **Semantic:** e.g., animal ~ cat
-

Methods of Intrinsic Evaluation

Qualitative Evaluation

- Visualize the embedding space: use **PCA** or **t-SNE** to project high-dimensional vectors (~100 dimensions) into **2D or 3D**
- Inspect **nearest neighbors** of selected words

- Analyze **clusters** of similar words

Quantitative Evaluation

- Compare word embedding similarity with **human judgment**
 1. Collect datasets of word similarity scores (e.g., **WordSim353**)
 2. Compute similarity of embedding vectors for word pairs
 3. Measure **correlation** between embedding similarity and human scores

Similarity Metrics

- **Cosine Similarity**: measures the angle between two vectors
- **L2 Distance (Euclidean)**: measures straight-line distance

Visualization

- Project embeddings using **PCA or t-SNE**
- Visualize clusters in 2D/3D
- Example: skip-gram trained on Wikipedia (1B tokens, FastText embeddings)

Applications of Evaluation

- **Downstream tasks**
 - **Word Sense Induction**: identify different senses of a word
 - **Semantic analysis**: track **semantic shifts** over time or across communities
-

Representing Documents into Vectors

Similar to word representations, documents can also be represented as vectors using:

1. **Word vectors**
 2. **Count-based representations**
 3. **Generative probabilistic models** (e.g., LDA)
 4. **Language models**
-

1. Using Word Vectors

- Given word vectors $x_1, \dots, x_n$, represent a document by a function:

$$f(x_1, \dots, x_n)$$

- Simple and common approach: **average word vectors**

$$\frac{1}{n} \sum_i x_i$$

2. Count-Based Representation (Bag-of-Words)

- Steps:
 1. Build **vocabulary** of the corpus
 2. Compute **document-level count matrix**

- Term Frequency (TF):**

$$tf_{t,d} = |\{t \in d\}|$$

- Result: each document = **vector of word counts**
- Bag-of-Words assumption:**
 - Ignores **word order**

Limits of Count-Based Representation

- Sensitive to:
 - **Frequent words** (e.g., “the”) → not informative but dominant
 - **Rare words** → may overly influence representation

Solution: TF-IDF

- Weight terms using **Inverse Document Frequency (IDF)**:

$$idf_{t,c} = \log \left(\frac{|C|}{|\{d \in C \text{ s.t. } t \in d\}|} \right)$$

- Idea:
 - Words in **many documents** → **low importance**
 - Words in **few documents** → **high importance**
 - Log used to **smooth impact**
- Final representation:

$$tf \times idf(t, d, C)$$

Dimensionality Reduction

- Apply techniques like **SVD**
- Leads to **Latent Semantic Analysis (LSA)**
- Produces **dense, lower-dimensional document vectors**

❖ MCQ

1. How can a word be defined semantically?
 - A. Any sequence of letters
 - B. Smallest linguistic expression with conventional non-compositional meaning that can be articulated in isolation
 - C. Only nouns and verbs
 - D. A token longer than one character**Answer: B**
2. What is the main goal of representing text with vectors?
 - A. Translate text
 - B. Identify a word, capture similarities, and help solve downstream tasks
 - C. Replace grammar rules
 - D. Generate random text**Answer: B**
3. Vector-based representations of text are called:
 - A. Synsets
 - B. Embeddings
 - C. POS tags
 - D. Tokens**Answer: B**
4. Traditional word representation assigns:
 - A. A vector based on meaning
 - B. A unique integer to each word
 - C. A semantic label
 - D. A frequency count only**Answer: B**
5. One-hot encoding of words assumes:
 - A. Words are semantically related
 - B. Word vectors form an orthonormal basis (orthogonal and normalized)
 - C. Words share embeddings
 - D. Polysemy is handled**Answer: B**
6. Problems with one-hot encoding include:
 - A. Words cannot be indexed
 - B. Words are considered equidistant, and polysemy is ignored
 - C. Words cannot be counted
 - D. It captures semantics perfectly**Answer: B**
7. Example of a morphological hand-crafted feature:
 - A. Synonym
 - B. Prefix, suffix, stem
 - C. Hypernym
 - D. Gloss**Answer: B**
8. Example of a grammatical hand-crafted feature:
 - A. Digit
 - B. Gender, number, part of speech
 - C. Synset ID
 - D. Hyponym**Answer: B**
9. Example of a shape-based hand-crafted feature:
 - A. Gloss
 - B. Capitalization, digit, hyphen
 - C. Hypernym

D. Morphology

Answer: B

10. Hand-crafted semantic features can be defined based on relations to other words, such as:

- A. Frequency
- B. Synonyms, hypernyms, antonyms
- C. Token index
- D. Word length

Answer: B

11. WordNet is:

- A. A neural network for words
- B. A large lexical database of word senses
- C. A one-hot encoding scheme
- D. A POS tagger

Answer: B

12. In WordNet, a word sense is:

- A. A unique integer
- B. A discrete representation of one aspect of a word's meaning
- C. Only a noun
- D. A character sequence

Answer: B

13. Word types in WordNet are grouped into:

- A. POS tags
- B. Synsets (cognitive synonym sets)
- C. Tokens
- D. Embeddings

Answer: B

14. A polysemous word in WordNet:

- A. Has only one synset
- B. Is assigned to multiple synsets
- C. Cannot appear in WordNet
- D. Is always a noun

Answer: B

15. WordNet organizes noun and verb synsets hierarchically to capture:

- A. Similarity scores only
- B. IS-A relations
- C. Word frequency
- D. Token sequences

Answer: B

16. A hyponym relationship is:

- A. X is a part of Y
- B. X is an instance of Y
- C. X is unrelated to Y
- D. Y is an instance of X

Answer: B

17. A hypernym relationship is:

- A. X is part of Y
- B. Y is an instance of X
- C. X is unrelated to Y
- D. X and Y are synonyms

Answer: B

18. Co-hyponyms are:

- A. Words sharing the same hypernym
- B. Words with the same spelling
- C. Words with different meanings

D. Words unrelated in WordNet

Answer: A

19. A meronymy relationship is:

- A. X is a part of Y
- B. X is a type of Y
- C. X is a synonym of Y
- D. X and Y are unrelated

Answer: A

20. A holonymy relationship is:

- A. X is part of Y
- B. Y is part of X
- C. X and Y are synonyms
- D. X is unrelated to Y

Answer: B

21. Similarity between synsets is higher when:

- A. The hypernym/hyponym path between them is shorter
- B. Synsets have different POS
- C. Synsets are far in the hierarchy
- D. Words are polysemous

Answer: A

22. Example similarity measure question: $\text{sim}(\text{dog}, \text{cat})$ is based on:

- A. One-hot vectors
- B. Hypernym/hyponym path length in WordNet
- C. Token frequency
- D. POS tags

Answer: B

23. Similarity between two synsets S_1 and S_2 is calculated as:

- A. $\text{sim}(S_1, S_2) = \text{length}(\text{path}(S_1, S_2))$
- B. $\text{sim}(S_1, S_2) = \frac{1}{\text{length}(\text{path}(S_1, S_2))}$
- C. $\text{sim}(S_1, S_2) = \max(S_1, S_2)$
- D. $\text{sim}(S_1, S_2) = \min(S_1, S_2)$

Answer: B

24. Similarity between two words w_1 and w_2 is defined as:

- A. Average similarity between all synsets of w_1 and w_2
- B. Maximum similarity between any synset of w_1 and any synset of w_2
- C. Sum of all context occurrences
- D. One-hot vector dot product

Answer: B

25. Subjective statements express:

- A. Facts that are verifiable
- B. Personal opinions, feelings, or beliefs
- C. Word frequencies
- D. Word embeddings

Answer: B

26. Objective statements describe:

- A. Personal beliefs
- B. Facts that are verifiable
- C. Word senses

D. Contexts only

Answer: B

27. The Distributional Hypothesis (Firth, 1957) states:

- A. Words have fixed meanings independent of context
- B. You shall know a word by the company it keeps
- C. Words are defined by their frequency only
- D. Polysemy cannot be solved

Answer: B

28. According to the Distributional Hypothesis, words appearing in similar contexts tend to:

- A. Be unrelated
- B. Have similar meanings
- C. Have the same POS tag only
- D. Always be synonyms

Answer: B

29. Contextual word representations are built by:

- A. Counting letters only
- B. Observing the words surrounding a target word
- C. Assigning random vectors
- D. Using only one-hot encoding

Answer: B

30. Example inference from context ("bardiwac") shows:

- A. Context helps determine the word's meaning
- B. Words cannot be inferred
- C. Only polysemous words matter
- D. One-hot vectors are sufficient

Answer: A

31. Steps to build distributional word vectors include:

- A. Define word context, count occurrences, build vectors from counts
- B. Assign random integers to words
- C. Use only POS tags
- D. Translate all words to another language

Answer: A

32. Contextual embeddings equation: $x_w = f(w, \text{Context}(w))$. Here, x_w represents:

- A. POS tag
- B. Contextual embedding vector of the word
- C. Synset ID
- D. One-hot encoding

Answer: B

33. In $x_w = f(w, \text{Context}(w))$, $\text{Context}(w)$ refers to:

- A. The word itself
- B. Surrounding words before and after w in a sentence
- C. Only the first word of the sentence
- D. Frequency counts only

Answer: B

34. The function f in contextual embeddings typically:

- A. Ignores context
- B. Maps the word and its context to a vector using a model like Transformer or Bi-LSTM
- C. Assigns a one-hot vector
- D. Selects a synset randomly

Answer: B

35. Contextual embeddings help solve which NLP problem?

- A. Polysemy (words with multiple meanings)
- B. Spelling errors
- C. One-hot encoding limitations only
- D. Tokenization

Answer: A

36. The context of a word can be defined as:

- A. Only the word itself
- B. Surrounding words, all words in sentence/paragraph, or preprocessed words
- C. Only nouns in the sentence
- D. Only hypernyms in WordNet

Answer: B

37. Two main approaches to model context for word vectors are:

- A. One-hot and WordNet
- B. Count-based and Prediction-based
- C. POS tagging and NER
- D. Synsets and hyponyms

Answer: B

38. Counting word occurrences in the context of a target word is used to:

- A. Generate POS tags
- B. Build co-occurrence matrices for word representation
- C. Create one-hot vectors only
- D. Detect polysemy

Answer: B

39. A co-occurrence matrix represents:

- A. Frequency of words appearing in the same context
- B. Syntactic dependencies
- C. One-hot vectors
- D. Named entities

Answer: A

40. A naive approach to vector representation of a word is:

- A. Take the column of the co-occurrence matrix
- B. Take the row of the co-occurrence matrix
- C. Use WordNet synsets
- D. Use POS tags only

Answer: B

41. Limits of naive co-occurrence-based representations include:

- A. Independence from corpus size
- B. Sensitivity to frequent and very infrequent words

- C. Captures polysemy perfectly
- D. One-hot vector equivalence

Answer: B

42. Pointwise Mutual Information (PMI) improves word representation by:

- A. Counting absolute co-occurrences
- B. Using relative probabilities of co-occurrences
- C. Ignoring context
- D. One-hot encoding words

Answer: B

43. PMI intuition: the more dependent two words are, the closer $P(\text{word1}, \text{word2})$ is to:

- A. 0
- B. $P(\text{word1}) + P(\text{word2})$
- C. $P(\text{word1})P(\text{word2})$
- D. 1-hot vector

Answer: C

44. Word embedding vectors using PMI typically use:

- A. Negative PMI values only
- B. Positive PMI values, Laplace smoothing, and log normalization
- C. Raw counts only
- D. Synset IDs

Answer: B

45. Advantages of PMI-based embeddings include:

- A. Dependence on corpus size
- B. Less sensitivity to frequent words due to log normalization
- C. No smoothing required
- D. Smaller vocabulary requirement

Answer: B

46. Limits of PMI-based embeddings include:

- A. Very large matrix size $O(V^2)$
- B. Easy comparison across corpora
- C. Low computational cost
- D. Independent of vocabulary size

Answer: A

47. Dimensionality reduction for PMI-based vectors can be done using:

- A. PCA only
- B. Singular Value Decomposition (SVD)
- C. One-hot encoding
- D. WordNet hierarchy

Answer: B

48. Steps in SVD-based word embeddings:

- A. Build a symmetric PMI matrix → Decompose with SVD → Use U (V, d) as word vectors
- B. Count words → Assign POS tags → Build vectors
- C. Use WordNet → Synset IDs → One-hot encoding
- D. Translate words → Count frequency → Build co-occurrence

Answer: A

49. Properties of SVD embeddings include:

- A. U is orthonormal
- B. Captures highest variance of original word embeddings
- C. Both A and B
- D. None of the above

Answer: C

50. Limits of dimensionality reduction approach (SVD) include:

- A. Requires storing matrix of size $O(V^2)$
- B. Computationally expensive $O(V * d^2)$
- C. Both A and B
- D. Works efficiently for extremely large corpora

Answer: C

51. Prediction-based word embeddings were introduced to:

- A. Avoid corpus dependence and large matrix computation
- B. Replace POS tagging
- C. Encode WordNet synsets only
- D. Reduce polysemy

Answer: A

52. In the Skip-Gram model, the main task is to:

- A. Predict a target word given its context
- B. Predict context words given a target word
- C. Count co-occurrences only
- D. Assign one-hot vectors only

Answer: B

53. In Word2Vec, the matrices **W** and **C** represent:

- A. Word and context embeddings
- B. POS and NER embeddings
- C. Co-occurrence counts
- D. SVD matrices

Answer: A

54. The objective function of Skip-Gram using Softmax is:

- A. $p(w | c) = \frac{e^{c \cdot w}}{\sum_v e^{v \cdot w}}$
- B. $p(c | w) = \frac{e^{w \cdot c}}{\sum_v e^{w \cdot v}}$
- C. $p(c | w) = \mathbf{w} \cdot \mathbf{c}$
- D. $p(c | w) = \sigma(\mathbf{w}, \mathbf{c})$

Answer: B

55. The main computational limitation of Softmax in Skip-Gram is:

- A. Complexity $O(d * K * T)$
- B. Memory bottleneck
- C. Complexity $O(V)$ to compute the loss per iteration
- D. It cannot handle negative sampling

Answer: C

56. Negative Sampling addresses the Softmax limitation by:

- A. Computing the probability over the full vocabulary
- B. Sampling K negative words not in the context ($K \ll V$)
- C. Ignoring context words
- D. Using one-hot vectors only

Answer: B

57. The Negative Sampling objective function is:

- A. $\log \sum_v e^{w \cdot v}$

B. $\sigma(\mathbf{w}, \mathbf{c}) + \frac{1}{K} \sum_{v \in N_K} \log \sigma(-\mathbf{w}, \mathbf{v})$

C. $\mathbf{w} \cdot \mathbf{c}$

D. $PMI(w, c)$

Answer: B

58. In the Negative Sampling formula, $\sigma(\mathbf{w}, \mathbf{c})$ represents:

- A. Probability that w and c co-occur
- B. Probability that w and c do not co-occur
- C. The negative sample
- D. The learning rate

Answer: A

59. In the Negative Sampling formula, $\log \sigma(-\mathbf{w}, \mathbf{v})$ represents:

- A. Maximizes probability of focus-context pairs
- B. Minimizes probability of focus-negative pairs
- C. Counts co-occurrences
- D. Computes Softmax

Answer: B

60. Complexity of Skip-Gram with Negative Sampling is:

- A. $O(V)$
- B. $O(d * K * T)$
- C. $O(V^2)$
- D. $O(d * V * T)$

Answer: B

61. Skip-Gram with Negative Sampling is mathematically equivalent to:

- A. One-hot encoding
- B. Computing the shifted-PMI matrix
- C. SVD on co-occurrence matrix
- D. CBOW

Answer: B

62. Limitations of Word2Vec Skip-Gram include:

- A. Fixed vocabulary
- B. Cannot model polysemy
- C. Cannot capture morphology
- D. All of the above

Answer: D

63. Extensions or variants of Skip-Gram include:

- A. CBOW
- B. GloVe
- C. FastText
- D. All of the above

Answer: D

64. Word2Vec embeddings are still useful because:

- A. They work well with deep learning architectures like LSTM
- B. They solve polysemy completely
- C. They automatically handle morphology
- D. They replace contextualized models like BERT

Answer: A

65. Extrinsic evaluation of word embeddings involves:

- A. Measuring vector similarity between words
- B. Using embeddings in a task-specific model and measuring task performance
- C. Visualizing embeddings in 2D or 3D
- D. Computing cosine similarity

Answer: B

66. Intrinsic evaluation of word embeddings is based on the idea that:

- A. Embeddings should be used only in downstream tasks
- B. Similar words should have similar vectors
- C. Word embeddings must be sparse
- D. Negative sampling improves embeddings

Answer: B

67. Which of the following is an example of morphological similarity?

- A. animal – cat
- B. computer – computers
- C. determiners – adjectives
- D. Paris – France

Answer: B

68. Which of the following is an example of syntactic similarity?

- A. computer – computers
- B. determiners – articles
- C. animal – cat
- D. Paris – France

Answer: B

69. Which of the following is an example of semantic similarity?

- A. cat – dog
- B. the – a
- C. running – ran
- D. table – tables

Answer: A

70. Qualitative evaluation of word embeddings includes:

- A. Computing cosine similarity between vectors
- B. Using embeddings in downstream tasks
- C. Visualizing embeddings and looking at nearest neighbors
- D. Measuring correlation with human judgment

Answer: C

71. Quantitative evaluation of word embeddings includes:

- A. Projecting embeddings with PCA
- B. Visual inspection of clusters
- C. Comparing vector similarity with human judgment
- D. Counting word co-occurrences

Answer: C

72. To visualize high-dimensional word embeddings, common techniques include:

- A. Softmax and Negative Sampling
- B. PCA and t-SNE
- C. SVD and CBOW
- D. Skip-Gram only

Answer: B

73. Common measures of similarity in the word embedding space are:

- A. Cosine similarity and L2 distance
- B. Dot product and Softmax

- C. Negative Sampling only
- D. PMI and co-occurrence counts

Answer: A

74. Steps to evaluate embeddings against human judgment include:

- A. Collect human judgment on word pairs, compute similarity of vectors, measure correlation
- B. Train Skip-Gram, compute negative samples, update weights
- C. Visualize vectors using PCA and check clusters
- D. Use embeddings in downstream tasks only

Answer: A

75. Applications of word embeddings include:

- A. Downstream NLP tasks
- B. Word Sense Induction
- C. Semantic analysis (e.g., semantic shift over time)
- D. All of the above

Answer: D

76. Documents can be represented as vectors using:

- A. Word vectors
- B. Count-based representations
- C. Generative models like LDA
- D. All of the above

Answer: D

77. A simple way to represent a document using word vectors is:

- A. Taking only the first word vector
- B. Summing or averaging the word vectors
- C. Using one-hot encoding
- D. Using POS tags

Answer: B

78. The function $f(x_1, \dots, x_n) = \frac{1}{n} \sum x_i$ represents:

- A. Maximum pooling
- B. Averaging word vectors
- C. TF-IDF weighting
- D. SVD decomposition

Answer: B

79. In count-based document representation, the first step is:

- A. Apply SVD
- B. Get the vocabulary of the corpus
- C. Compute embeddings
- D. Perform tokenization only

Answer: B

80. The term-frequency (TF) is defined as:

- A. Number of documents containing a term
- B. Number of times a term appears in a document
- C. Probability of a word
- D. Similarity score

Answer: B

81. A bag-of-words model assumes:

- A. Word order matters
- B. Word order does not matter
- C. Only verbs are important
- D. Only nouns are counted

Answer: B

82. A limitation of count-based representations is:

- A. Insensitivity to frequent words
- B. High sensitivity to frequent and rare words
- C. No vocabulary needed
- D. No computation required

Answer: B

83. A word appearing in all documents (e.g., “the”) is:

- A. Very informative
- B. Not informative for document representation
- C. Rare
- D. Ignored automatically

Answer: B

84. A word appearing in only one document is:

- A. Not useful
- B. Likely to be informative
- C. Always removed
- D. Always frequent

Answer: B

85. Inverse Document Frequency (IDF) is used to:

- A. Count word occurrences
- B. Reduce the importance of frequent words
- C. Increase the importance of frequent words
- D. Remove rare words

Answer: B

86. The IDF formula is:

- A. $idf = \frac{|C|}{|d|}$
- B. $idf = \log \left(\frac{|C|}{|\{d \in C: t \in d\}|} \right)$
- C. $idf = tf * d$
- D. $idf = \sum tf$

Answer: B

87. Why is the logarithm used in IDF?

- A. To increase computation time
- B. To smooth the impact of rare words
- C. To remove frequent words
- D. To normalize vectors

Answer: B

88. TF-IDF combines:

- A. Term frequency and inverse document frequency

- B. Word embeddings and SVD
- C. POS tagging and parsing
- D. WordNet and PMI

Answer: A

89. After applying TF-IDF, dimensionality reduction can be done using:

- A. Softmax
- B. SVD (Latent Semantic Analysis)
- C. Negative Sampling
- D. Tokenization

Answer: B

90. Latent Semantic Analysis (LSA) is based on:

- A. Word2Vec
- B. SVD applied to TF-IDF matrix
- C. POS tagging
- D. Named Entity Recognition

Answer: B